

## Week 2: Context Free Grammars

- \*\* 1. Design CFGs for the following programming language lexemes over the ASCII alphabet. You will find it convenient to use abbreviations like  $\dots$  to help present the rules compactly.
- (a) A *C program identifier* is any string of length at least 1 containing only letters ('a'-'z', lower and uppercase), digits ('0'-'9') and the underscore, and which begins with a letter or the underscore.
  - (b) An *integer literal* is any string taking one of the following forms:
    - a non-empty sequence of digits (decimal)
    - a non-empty sequence of characters from '0'-'9', 'a'-'e' (upper or lowercase) that are preceded by "0x" (hexadecimal)
    - a non-empty sequence of bits '0' and '1' that are preceded by "0b" (binary)
- \*\* 2. Give a CFG that describes the language of all subsequences of all permutations (i.e. no repetition) of the following keywords:
- final static synchronized
- \* 3. Consider the following grammar, which describes the structure of statements in an imperative programming language.

|              |       |                                          |     |
|--------------|-------|------------------------------------------|-----|
| <i>Prog</i>  | $::=$ | <i>Stmt Stmt</i>                         | (1) |
| <i>Stmt</i>  | $::=$ | if exp then <i>Stmt</i> else <i>Stmt</i> | (2) |
|              |       | while exp do <i>Stmt</i>                 | (3) |
|              |       | skip                                     | (4) |
|              |       | id $\leftarrow$ exp                      | (5) |
|              |       | { <i>Stmt Stmt</i> }                     | (6) |
| <i>Stmts</i> | $::=$ | ; <i>Stmt Stmt</i>                       | (7) |
|              |       | $\epsilon$                               | (8) |

In it expressions appear only as a terminal symbols exp because the structure of expressions is not important to the exercises. In total, the terminal symbols are: if, then, else, while, do, skip, id, exp, the left and right braces, the end-of-input marker, the assignment backwards-arrow and the semicolon. The rules are numbered to make constructing the parse tables easier. The language of this grammar (start symbol *Prog*) includes strings such as:

while exp do id  $\leftarrow$  exp

i.e. strings that show the control structure of the program without specifying the particular expressions involved.

The nullable, first and follow maps for the nonterminals in this grammar are as follows:

| Nonterminal | Nullable? | First                  | Follow     |
|-------------|-----------|------------------------|------------|
| Prog        | ×         | if, while, skip, id, { |            |
| Stmt        | ×         | if, while, skip, id, { | else, ;, } |
| Stmts       | ✓         | ;                      | }          |

- For each rule  $X ::= \alpha$  numbered (1) – (8), compute  $\text{First}(\alpha)$ , the set of terminal symbols that can start any string derivable from the rule right-hand side  $\alpha$ .
- Construct the parsing table for the grammar.
- Is the grammar LL(1)?

\* 4. Consider now the following grammar:

|              |       |                                           |     |
|--------------|-------|-------------------------------------------|-----|
| <i>Prog</i>  | $::=$ | <i>Stmt Stmts</i>                         | (1) |
| <i>Stmt</i>  | $::=$ | if bexp then <i>Stmt</i> else <i>Stmt</i> | (2) |
|              |       | while bexp do <i>Stmt</i>                 | (3) |
|              |       | skip                                      | (4) |
|              |       | id $\leftarrow$ <i>aexp</i>               | (5) |
|              |       | <i>Stmt Stmts</i>                         | (6) |
| <i>Stmts</i> | $::=$ | ; <i>Stmt Stmts</i>                       | (7) |
|              |       | $\epsilon$                                | (8) |

This grammar is the same as the previous one, except that braces have been removed in rule 6. The Nullable, First and Follow maps for this grammar can be tabulated as follows.

| Nonterminal | Nullable? | First               | Follow  |
|-------------|-----------|---------------------|---------|
| Prog        | ×         | if, while, skip, id |         |
| Stmt        | ×         | if, while, skip, id | ;, else |
| Stmts       | ✓         | ;                   | ;, else |

- For each rule  $X ::= \alpha$  numbered (1) – (8), compute  $\text{First}(\alpha)$ , the set of terminal symbols that can start any string derivable from the rule right-hand side  $\alpha$ .
- Construct the parsing table for this grammar.
- Is the grammar LL(1)?

\*\* 5. Consider the following grammar for arithmetic expressions:

$$\begin{aligned}
 E &::= \text{num} & (1) \\
 &| \text{id} & (2) \\
 &| E + E & (3) \\
 &| E * E & (4) \\
 &| (E) & (5)
 \end{aligned}$$

The 6 terminal symbols of this grammar are:

num id + \* ( )

The nullable, first and follow sets for the nonterminals are:

| Nonterminal | Nullable? | First      | Follow  |
|-------------|-----------|------------|---------|
| E           | ×         | num, id, ( | +, *, ) |

- For each of the rules (1) – (5), construct the first set of the right-hand side of the rule.
- Construct the parsing table for this grammar.
- What do the rule conflicts (cells of the parsing table containing more than one rule) of this grammar and the previous grammar have in common? Without constructing the parsing table, argue that the following grammar is sure to have a rule conflict:

$$\begin{aligned}
 X &::= a \\
 &| b \\
 &| X c
 \end{aligned}$$

\* 6. Consider the following grammar for arithmetic expressions, whose terminals are as above:

$$\begin{aligned}
 E &::= \text{num } F \\
 &| \text{id } F \\
 &| ( E ) F \\
 F &::= + E \\
 &| * E \\
 &| \epsilon
 \end{aligned}$$

The nullable, first and follow sets can be described by:

| Nonterminal | Nullable? | First      | Follow |
|-------------|-----------|------------|--------|
| E           | ×         | num, id, ( | )      |
| F           | ✓         | +, *       | )      |

- Construct the first set of each of the right-hand sides of rules in the grammar.
- Construct the parsing table of the grammar.
- Is the grammar LL(1)?

\*\* 7. Give CFGs for the following languages. The later parts are more difficult than 2-star.

- (a) All odd length strings over  $\{a, b\}$ .
- (b) All strings over  $\{a, b\}$  that contain  $aab$  as a substring.
- (c) The set of strings over  $\{a, b\}$  with more  $a$  than  $b$ . Hint: every string  $w$  with at least as many  $a$  as  $b$  (possibly the same number of  $a$  as  $b$ ) can be characterised inductively as follows. Either:
  - $w$  is just  $a$
  - or,  $w$  is of shape  $avb$  with  $v$  containing at least as many  $a$  as  $b$
  - or,  $w$  is of shape  $bva$  with  $v$  containing at least as many  $a$  as  $b$
  - or,  $w$  is of shape  $v_1v_2$  with  $v_1$  and  $v_2$  each separately containing at least as many  $a$  as  $b$
  - or,  $w$  is the empty string
- (d) The complement of the language  $\{a^n b^n \mid n \geq 0\}$  over  $\{a, b\}$ . Hint: express "not of shape  $a^n b^n$  for some  $n$ " into one or more positive (i.e. without using *not* or similar) conditions.
- (e)  $\{v\#w \mid v, w \in \{a, b\}^* \text{ and the reverse of } v \text{ is a substring of } w\}$ , over  $\{a, b, \#\}$ .